



Trailhead



[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Migrate JavaScript

Migrate JavaScript

Learning Objectives

After completing this unit, you'll be able to:

- Migrate JavaScript from Aura components to Lightning web components.
- Share code between Aura components and Lightning web components.

Use JavaScript Modules

JavaScript is the beating heart of a Lightning web component. Without JavaScript, a component is a lifeless lump of mainly HTML. We got a glimpse of the JavaScript file in a Lightning web component in the previous unit when we saw how attributes in an Aura component map to JavaScript properties. Now we dig deeper into the JavaScript file.

Migrate JavaScript code from the separate client-side controller, helper, and renderer files in an Aura component to one JavaScript file in a Lightning web component.

A client-side controller in an Aura component is a JavaScript object in object-literal notation containing a map of name-value pairs. That's a lot of words. Let's look at an example in the `PropertyPaginatorController.js` file:

```
1  ({
2    previousPage : function(component) {
3      var pageChangeEvent =
4      component.getEvent("pagePrevious");
5      pageChangeEvent.fire();
6    },
7    nextPage : function(component) {
8      var pageChangeEvent = component.getEvent("pageNext");
9      pageChangeEvent.fire();
```



Hi NELLURI, ask me about products, features, and pricing, or connect to a sale...





JavaScript you write looks similar to the JavaScript you'd write for any other modern JavaScript framework.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers/trailmix_creator_id=sreb7&trailmix_slug=salesforce-platform-developer\)](#)

Migrate JavaScript

```
1 import { LightningElement, api } from 'lwc';
2 export default class Paginator extends LightningElement {
3     /** The current page number. */
4     @api pageNumber;
5     /** The number of items on a page. */
6     @api pageSize;
7     /** The total number of items in the list. */
8     @api totalItemCount;
9     previousHandler() {
10         this.dispatchEvent(new CustomEvent('previous'));
11     }
12     nextHandler() {
13         this.dispatchEvent(new CustomEvent('next'));
14     }
15     get currentPageNumber() {
16         return this.totalItemCount === 0 ? 0 : this.pageNumber;
17     }
18     get isFirstPage() {
19         return this.pageNumber === 1;
20     }
```



Show More

We won't look at each line of code. The main takeaway is that Lightning web components use standard JavaScript modules. That's a good thing!

Share Code Between Aura Components and Lightning Web Components

To share JavaScript code between Lightning web components and Aura components,





```

3   * Returns whether provided value is a function
4   * @param {*} value - the value to be checked
5   * @return {boolean} true if the value is a function, false
6   * otherwise
7   */
8   export function isFunction(value) {
9       return typeof value === 'function';
10  }

```

Migrate JavaScript

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

The utils module exports an isFunction() function that returns whether the provided parameter is a function.

Here's a c-libcaller Lightning web component that uses the utils module.

```

1  /* libcaller.js */
2  import { LightningElement } from 'lwc';
3  // import the library
4  import { isFunction } from 'c/utils';
5  export default class LibCaller extends LightningElement {
6      result;
7      checkType() {
8          // Call the imported library function
9          this.result = isFunction(
10             function() {
11                 console.log('I am a function');
12             }
13         );
14     }
15 }

```

The c-libcaller component imports the utils module and calls the isFunction function exported by the module. The argument passed into isFunction is a function so `result` is set to true.

Now, let's look at a libcallerAura.cmp file that also uses the utils module.

```
1 <aura:component>
```





The `localAura` Aura component includes a reference to the `utils` ES6 module. We added an `aura:id` so that we can get a reference to the module in JavaScript code.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#). Here's the `localAuraController.js` file.

Migrate JavaScript



```
2      doInit: function(cmp) {
3          // Call the lib here
4          var libCmp = cmp.find('utils');
5          var result = libCmp.isFunction(
6              function() {
7                  console.log(" I am a function");
8              }
9          );
10         console.log("Is it a function?: " + result);
11     }
12 })
```

The component's controller uses `cmp.find('utils')` to match the `aura:id` in markup and get a reference to the module. The `utils` ES6 module exports `isFunction()`, which the Aura component calls. The argument passed into `isFunction()` is a function so `result` is set to `true`.

Use Third-Party JavaScript Libraries

To use a third-party JavaScript library in an Aura component or a Lightning web component, you must upload the library as a static resource. The syntax for working with the static resource is different in the two programming models.

In an Aura component, use the `<ltng:require>` tag in markup to load the static resource.

```
1 <ltng:require scripts="{!$Resource.resourceName}"
2     afterScriptsLoaded="{!c.afterScriptsLoaded}" />
```



`resourceName` is the name of the static resource. The `afterScriptsLoaded` action in the client-side controller is called after the scripts are loaded and the component is





For details on using the imported library, see [Use Third-Party JavaScript Libraries](https://developer.salesforce.com/docs/component-lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) (https://developer.salesforce.com/docs/component-lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).
[Lightning Web Components for Aura Developers \(content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://developer.salesforce.com/docs/component-lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
[Components Developer Guide](https://developer.salesforce.com/docs/component-lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

Migrate JavaScript

Dynamically Creating Components

In an Aura component, you can dynamically create a component in JavaScript using `$A.createComponent()`. There's no equivalent for dynamically creating a component in a Lightning web component.

This architectural decision is deliberate. The main reason for not replicating this pattern in Lightning web components is that the pattern in Aura components has led to buggy and convoluted code.

A better alternative in Lightning web components is to create multiple HTML templates for your component. The component's `render()` method can switch templates based on the component needs. This pattern is more similar to the route splitting used in other JavaScript frameworks.

An Opportunity to Refactor

As you migrate JavaScript code, notice that it's not a line-by-line conversion. It's a good opportunity to revisit your component's design.

Migrating to a new programming model is like taking a new path that leads you to the same destination. The sights and sounds (and code you write) along the way are different.

Migrating a component is also a great opportunity to use fancy new ES6 features, such as JavaScript modules, that you probably aren't using in Aura components.

The [Modern JavaScript Development](https://trailhead.salesforce.com/en/content/learn/modules/modern-javascript-development) (<https://trailhead.salesforce.com/en/content/learn/modules/modern-javascript-development>) Trailhead module gives you a great overview of these new features.

Resources





Quiz Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).
To complete this unit, you need to answer all the quiz questions correctly.

Migrate JavaScript +100 Points

1 What format is a JavaScript file in a Lightning web component?

- ☐ A A JSON file
- ☐ B An ES6 module
- ☐ C An XML file
- ☐ D An HTML:JavaScript mashup file

2 How do you access an ES6 module in an Aura component?

- ☐ A Use an import statement.
- ☐ B Add an attribute that references the module.





Second attempt earns 50 points. Three or more earns 25 points.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Migrate JavaScript

Learn

Badges
Trails
Trailmixes
Superbadges
Trailhead Academy
Career Paths

Certifications

Certifications
Maintain Certifications
Verify Certifications

Community

Trailblazer Community
Events
Agentblazers
Salesforce Developers
Salesforce Admins
Code of Conduct
Report Illegal Content (EU)

Extras

Trailhead Login
Sales Enablement
Trailblazer Store
Salesforce Help

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)

Engli

Your Privacy Choices

